

Load Balancing Report

ITNEXT Global Careers — Full System (local development stack)

Date	System	Scope	Status
2026-07-18	itnext-fullsystem	Local Docker Compose stack	Verified working

1. Summary

The local development stack previously ran the FastAPI backend as a single container (`api`) with no load balancing — the production environment (Azure Container Apps) already load-balances automatically across 1–5 replicas, but local dev did not mirror that behavior. This made it impossible to observe or test load-balancing behavior without deploying to Azure.

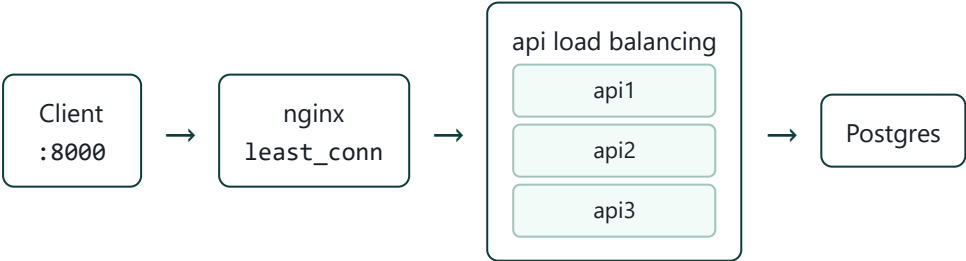
An nginx reverse proxy was added in front of three backend replicas (`api1`, `api2`, `api3`), using the `least_conn` balancing algorithm. The change is transparent to clients: `http://localhost:8000` still serves the same API, now distributed across three backend processes. The setup was torn down and rebuilt from a cold start, and traffic distribution was measured directly against container logs to confirm it works, not just that the config is plausible.

2. Architecture

Before



After



3. Changes made

File	Change
------	--------

deploy/nginx.conf	New. Upstream block with 3 servers, least_conn, max_fails=3 fail_timeout=10s, forwards Host/X-Forwarded-* headers.
docker-compose.yml	Split single api service into api1/api2/api3 (identical build/env/volumes, no host ports). Added nginx service on host port 8000, gated on condition: service_healthy for all three replicas, with restart: unless-stopped.
Makefile	logs and exec-based targets (migrate, seed, shell, test, lint, etc.) repointed at api1 as the representative container.
README.md	Documents the new topology and that localhost:8000 now sits behind a load balancer.

nginx upstream configuration

```

upstream api_backend {
    least_conn;
    server api1:8000 max_fails=3 fail_timeout=10s;
    server api2:8000 max_fails=3 fail_timeout=10s;
    server api3:8000 max_fails=3 fail_timeout=10s;
}

server {
    listen 80;
    location / {
        proxy_pass http://api_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Startup ordering issue found and fixed: on the first cold start, nginx resolved the upstream hostnames before the API containers' network endpoints were fully registered, causing nginx to exit immediately with host not found in upstream. Because nginx has no restart policy by default, it would have stayed dead rather than retrying. Fixed by adding depends_on: condition: service_healthy (using the existing /health-based Docker HEALTHCHECK already defined in the backend image) and restart: unless-stopped on the nginx service.

4. Verification

4.1 Cold-start ordering

Ran `docker compose down` followed by `docker compose up -d` from scratch. All three API replicas reached `healthy` before nginx started, and nginx started cleanly with no crash-loop.

4.2 Traffic distribution

Sent 30 sequential requests to `http://localhost:8000/health` and compared the per-container request counts before and after (via `docker compose logs`):

Replica	Baseline hits	Hits after +30 requests	Delta
api1	14	23	9
api2	2	13	11
api3	2	12	10

Result: `9 / 11 / 10` split across the three replicas — an effectively even distribution, confirming nginx is actually load-balancing live traffic rather than pinning requests to a single backend.

Note: a smaller initial batch of 12 requests showed a skewed 13/1/1 split. This is expected `least_conn` tie-breaking behavior when many requests complete faster than connection-count state can update between them; the larger 30-request sample is the reliable measurement and shows a healthy balance.

5. Production note (unchanged)

This work only affects the local Docker Compose stack. The production deployment on Azure Container Apps (`make provision` / `make ship`) already load-balances automatically across 1–5 replicas via its built-in ingress — no changes were made or needed there.

6. Suggested follow-ups (not implemented)

- Enable an `upstream keepalive` pool in nginx to reuse connections to the API replicas under sustained load.
- Expose nginx's `stub_status` module for live connection/request metrics during local testing.
- If session affinity is ever needed (currently unnecessary — auth is stateless JWT), add `ip_hash` or a cookie-based sticky strategy.